

Лабораторна робота № 6

«Дослідження арифметичних пристроїв»

1. Теоретичні відомості.

Короткі теоретичні відомості з мови Verilog (продовження).

Підключення модулів у файлі верхнього рівня ієрархії.

Основною проектною одиницею на мові Verilog є модулі. При створенні складних пристроїв зручно використовувати ієрархічний підхід до їх побудови, тобто до складу одного пристрою може входити кілька модулів. Прийшов час навчитися підключати готові модулі до спроектованої схеми і пов'язувати їх між собою. Зазвичай, вихід одного модуля з'єднується з входом іншого провідником, тому зміни вихідного сигналу джерела будуть негайно передаватися на вхід приймача. Можна з'єднувати порти вводу/виводу модуля з входами пристрою. Зв'язки між модулями можуть бути типів **wire** та **reg**.

Синтаксис підключення модуля має наступний вигляд:

module_name instance_name (... io ports ...);

де ***module_name*** - ім'я викликаемого модуля, який необхідно підключити. Можна, звичайно, підключити в одному проекті одночасно декілька однакових модулів, просто вказавши для них різні імена - ***instance_name***. Тут доречно згадати механізм оголошення змінних в мовах програмування. Спочатку записується тип змінної, а потім - ім'я змінної. Так і при підключенні модуля - спочатку записується ім'я модуля, який необхідно підключити, а потім ім'я конкретного екземпляра цього модуля в файлі, де він використовується. Далі, в круглих дужках, зазначається підключення сигналів (типу **reg** або **wire**) до портів вводу/виводу модуля.

Розглянемо наступний приклад:

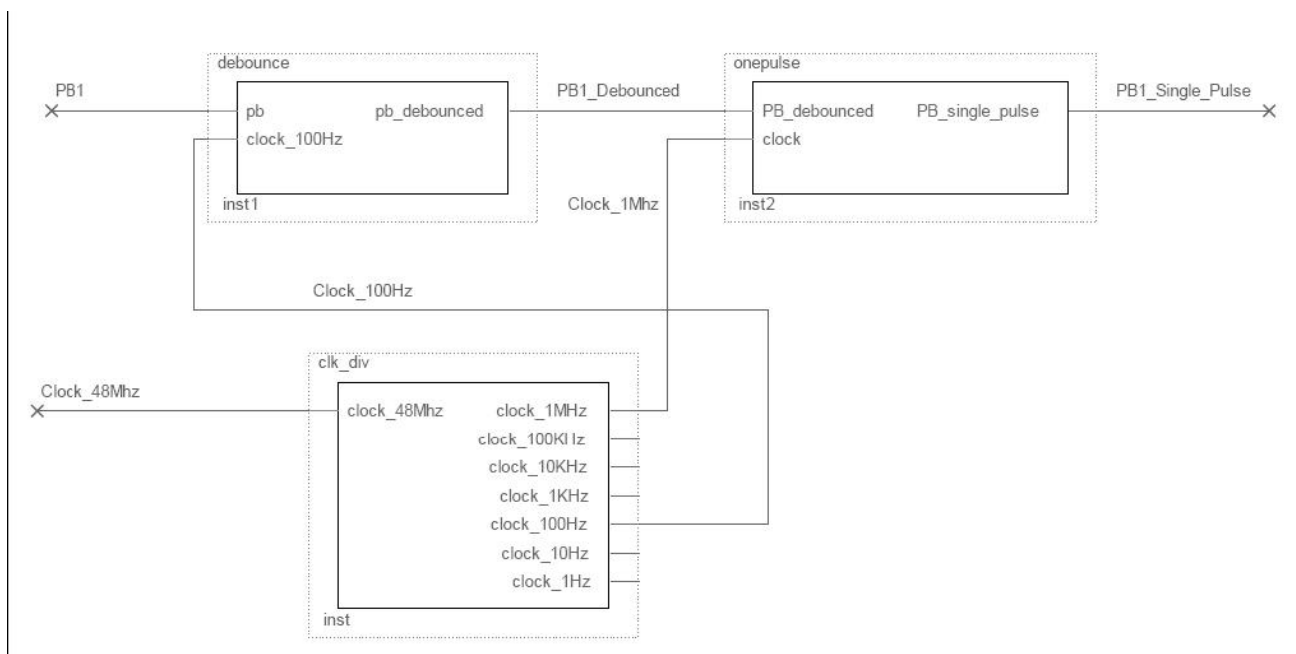


Рис. 1. Ієрархічне підключення модулів

Припустимо, спроектований пристрій містить в собі три модулі (рис.1): **debounce**, **onepulse**, **clk_div**, кожен з яких описаний у відповідному файлі на мові Verilog (назва файлу має збігатися з ім'ям модуля). Відповідні вхідні і вихідні порти цих модулів зображені на малюнку. Вхід модуля зображується з лівого боку блоку, вихід - з правого. У

пристрої необхідно об'єднати між собою ці модулі в файлі верхнього рівня ієрархії. З'єднання модулів здійснюються за допомогою сигналів (типу **reg**) -**Clock_100Hz**, **Clock_1MHz**, **PB1_Debounced**. Файл опису пристрою буде виглядати наступним чином:

```
module hierarch (Clock_48MHz, PB1, PB1_Single_Pulse);  
input Clock_48MHz, PB1;  
output PB1_Single_Pulse;  
  
reg Clock_100Hz, Clock_1MHz, PB1_Debounced;  
  
debounce debounce1 (PB1, Clock_100Hz, PB1_Debounced);  
clk_div clk_div1 (Clock_48MHz, Clock_1MHz, Clock_100Hz);  
onepulse onepulse1 (PB1_Debounced, Clock_1MHz, PB1_Single_Pulse);  
endmodule
```

Оператори блокуючого і неблокуючого присвоювання

У мові Verilog існують два типи операторів присвоювання: блокуючий **blocking** (=), і неблокуючий **nonblocking** (<=).

Для того щоб зрозуміти різницю між даними типами операторів присвоювання, необхідно розглянути принцип роботи Verilog симулятора. У реальному пристрої (наприклад - цифрової схеми), яка моделюється за допомогою мови Verilog, події можуть відбуватися одночасно - при зміні вхідного сигналу у всіх елементах, пов'язаних з ним, починаються відповідні процеси. Вони протікають одночасно і призводять до відповідних змін вихідних сигналів. Моделююча програма не може виконувати події одночасно, вона створює списки подій, які будуть виконуватися послідовно. Коли всі події зі списку виконані, симулятор переходить до обробки наступного часового кроку - збільшує поточний час моделювання на часовий інтервал (другий параметр в директиві **`timescale**) і виконує обробку списку подій, які повинні відбутися на даному етапі. Розглянуті події відбуваються «одночасно» - тобто на одному часовому кроці моделювання. Припустимо, є наступний набір команд:

```
always @ (Posedge CLK) a = b;  
always @ (Posedge CLK) b = a;
```

В даному прикладі змінні *a* і *b* - однорозрядні регістри, які до моменту появи зростаючого фронту тактового сигналу CLK зберігали наступне значення: *a* == 0 і *b* == 1. Яке ж значення матимуть ці змінні після виконання операції присвоєння? Це залежить від того, в якій послідовності операції присвоювання потраплять в список обробки. Тобто, поведінка такої конструкції залежить від порядку проходження операторів в програмі. Це означає, що або обидві змінні будуть дорівнювати 0, або обидві дорівнюватимуть 1 (в нашому прикладі - 1). Операція блокуючого присвоювання (=) блокує виконання інших послідовних операцій до тих пір, поки вона не буде виконана. Використання операції блокуючого присвоєння в блоках, що мають виконуватись паралельно, небажано. Але, якщо в блоці необхідно забезпечити послідовне виконання операторів, слід використовувати даний тип присвоєння.

Наступний фрагмент програми гарантує обнулення змінних *a* і *b* по передньому фронту сигналу CLK:

```
always @ (Posedge CLK)  
begin
```

```

a = 0;
b = a;
end

```

Якщо в попередньому прикладі використовувати оператор неблокуючого присвоєння ($\leq$), то поведінка пристрою зміниться:

```

always @ (Posedge CLK) a <= b;
always @ (Posedge CLK) b <= a;

```

В даному випадку, в списку подій, що виконуються на поточному часовому кроці моделювання після зміни сигналу CLK, обидві операції будуть розташовані як ті, що виконуються паралельно. Тобто, змінні a і b обмінюються своїми значеннями. Після проходження зростаючого фронту сигналу CLK значення змінних будуть наступними: $a == 1$ $b == 0$. Послідовність запису $a \leq b$; $b \leq a$; або $b \leq a$; $a \leq b$; в даному випадку не має значення, тому що події моделюються одночасно.

Системні функції мови Verilog

Мова Verilog надає програмісту більше доступних можливостей для управління та аналізу результатів моделювання. Ці можливості реалізовані у вигляді системних функцій симулятора. Слід пам'ятати, що при синтезі системні функції ігноруються.

Завдяки наявності механізму PLI, що забезпечує підключення додаткової програми (написаної або користувачем, або іншою стороною) до тестових файлів, число системних функцій і завдань, які можуть виконуватися за їх допомогою, суттєво зростає. Основне призначення системних функцій - збір і аналіз інформації, взаємодія з операційною системою. Ознакою системної функції є знак $\$$. Наведемо найбільш популярні системні функції:

\$finish - завершення процесу моделювання;

\$stop – призупинка моделювання та перехід в інтерактивний режим;

\$display, **\$write** - пересилка даних за допомогою функції **stdout** (дані дублюються в файлі протоколу). Поведінка такої функції відповідає функції **printf** мови C (видача форматowanego рядку з підтримкою додаткових форматів, наприклад, $\%b$ -бінарний), або процедури **write** мови Паскаль з розділеними « $\cdot$ » аргументами. Функція **\$display** завершує видачу рядка командою «новий рядок»;

\$monitor - відстежує зміни аргументів і в кінці кожного часового кроку моделювання відображає поточні результати (якщо були виявлені зміни значень сигналів). Формат даної функції - як у **\$display**;

\$readmemb, **\$readmemh** - забезпечують зчитування даних (в двійковому або шістнадцятковому форматі) з файлу в пам'ять пристрою. Формат файлу дуже простий - в кожному рядку вказується слово заданої розрядності, або покажчик адреси (конструкція **@<адреса завантаження>**). Дану функцію зручно застосовувати для моделювання ПЗУ;

\$system - виконує команду операційної системи (виклик функції мови C **system ()**).

Для виконання файлових операцій використовуються функції **\$fopen**, **\$fclose**, **\$fwrite**, **\$fmonitor**. Вони дозволяють зберігати дані, що передаються, в файлах. Функції **\$dumpfile**, **\$dumpvars** дозволяють записувати зміни сигналів модуля, що тестується, для всього проекту або його складових частин, у файлі спеціального формату для подальшого аналізу. Це дуже корисні й ефективні функції для обробки даних.

Функція **\$time** - повертає значення поточного часу моделювання.

Це невеликий перелік стандартних функцій. Їх повний список вказано в документації до моделюючої програми.

Арифметичні пристрої.

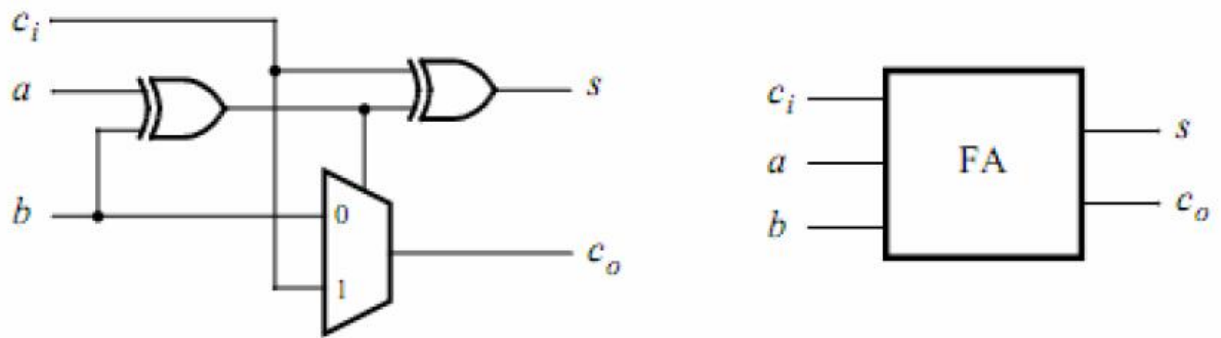


Рис. 2. Схема повного однорозрядного суматора і його графічне зображення.

Призначення сигналів суматора наступне: a і b - вхідні сигнали (складові), C_i - вхід переносу з попереднього розряду, s - вихідний сигнал (сума), C_o - вихід переносу до наступного розряду. Таблиця дійсності однорозрядного суматора виглядає наступним чином:

Таблиця істинності

a	b	Ci	Co	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Для створення сумматорів більшої розрядності використовується каскадне з'єднання однорозрядних суматорів. Наприклад, на рисунку 3 показано повний чотирьохрозрядний суматор з послідовним переносом, що складається з чотирьох однорозрядних суматорів.

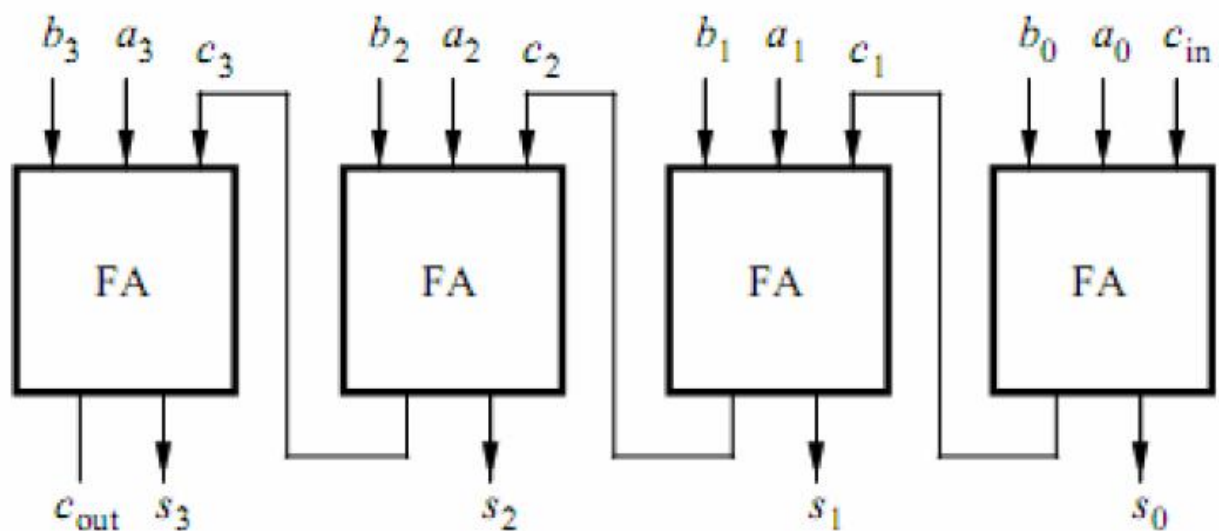


Рис. 3. Повний чотирьохрозрядний суматор з послідовним переносом.

Приклад виконання операції множення двох 4-розрядних чисел в двійковому вигляді показано на рисунку 4. На рисунку 5 показана структурна схема реалізації такого перемножувача. Він складається з логічних елементів «І» і повних однорозрядних суматорів (FA).

				a_3	a_2	a_1	a_0
			$\times$	b_3	b_2	b_1	b_0
				$a_3 b_0$	$a_2 b_0$	$a_1 b_0$	$a_0 b_0$
		$a_3 b_1$		$a_2 b_1$	$a_1 b_1$	$a_0 b_1$	
	$a_3 b_2$	$a_2 b_2$		$a_1 b_2$	$a_0 b_2$		
$a_3 b_3$	$a_2 b_3$	$a_1 b_3$	$a_0 b_3$				
p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

Рис. 4. Операція множення двох 4-розрядних чисел.

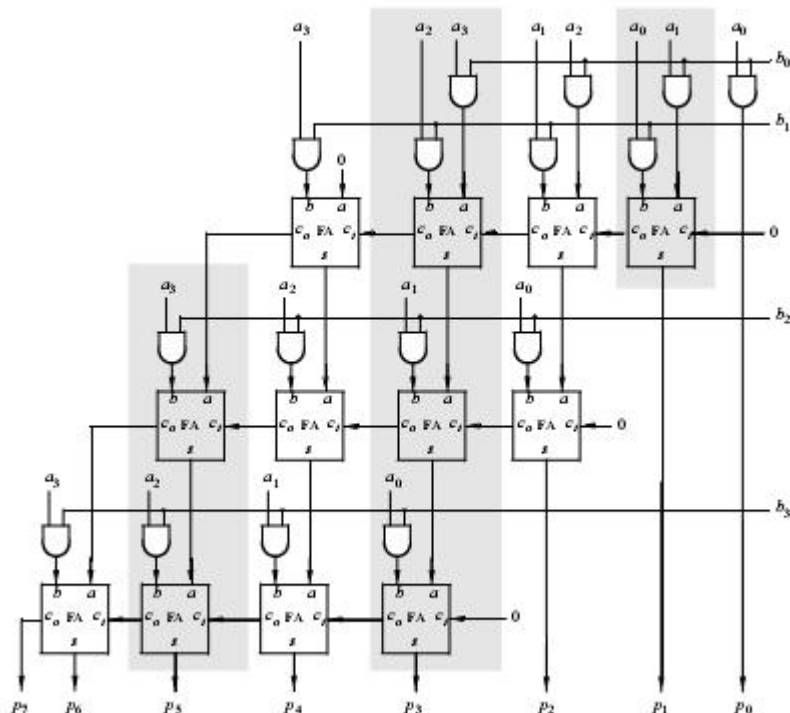


Рис. 5. Структурна схема перемножувача двох 4-розрядних чисел.

Згадані два компонента використовуються як базові для створення різних арифметичних або арифметико-логічних пристроїв (АЛП).

2. Порядок виконання роботи.

У лабораторній роботі ми створимо проект, що описує роботу повного чотирирозрядного суматора. Для головного модуля використовується структурний опис. Для перевірки суматора створимо еталонну модель, яка використовує поведінковий опис, і тестовий файл (test-bench). Для цього:

1. Створіть новий проект в середовищі ModelSim.
2. Створіть похідний файл суматора на структурному рівні:

```
module my_sum (Ain, Bin, Ci, Sout, Co);
```

```
  input Ain, Bin, Ci;
```

```
  output Sout, Co;
```

```
  wire [3: 0] Ain, Bin, Sout, C;
```

```
  wire Ci, Co;
```

```
  bitsum sum1 (Ain [0], Bin [0], Sout [0], Ci, C [0]);
```

```
  bitsum sum2 (Ain [1], Bin [1], Sout [1], C [0], C [1]);
```

```
  bitsum sum3 (Ain [2], Bin [2], Sout [2], C [1], C [2]);
```

```
  bitsum sum4 (Ain [3], Bin [3], Sout [3], C [2], C [3]);
```

```
  assign Co = C [3];
```

```
endmodule
```

```

module bitsum (A, B, S, Cin, Cout);
  input A, B, Cin;
  output S, Cout;

  wire A, B, S, Res;
  wire c1, c2, Cin, Cout;

  xor (Res, A, B);
  and(C1, A, B);
  xor(S, Cin, Res);
  and(C2, Cin, Res);
  or(Cout, c1, c2);

endmodule

```

Зверніть увагу на підключення готових модулів в даному файлі.

3. Створіть похідний файл еталонного суматора на поведінковому рівні:

```

module ref_sum (Ain, Bin, Ci, Sout, Co);
  input Ain, Bin, Ci;
  output Sout, Co;

  wire [3: 0] Sout, Ain, Bin;
  reg [4: 0] S;

  always @ (Ain, Bin, Ci)
    S = Ain + Bin + Ci;

  assign Sout = S [3: 0];
  assign Co = S [4];

endmodule

```

3. Створіть тестовий файл для подачі вхідних сигналів і порівняння роботи двох модулів:

```

module test_sum; // Top Level Testbench

  wire Ci, cm, cr;
  wire [3: 0] Ain, Bin;
  reg [3: 0] Ain_r, Bin_r;
  reg Ci_r;
  wire [3: 0] res_my, res_ref;

  my_sum my_block (Ain, Bin, Ci, res_my, cm);
  ref_sum ref_block (Ain, Bin, Ci, res_ref, cr);

  initial
  begin
    $display( "\ T \ t Time Ain Bin Ci res_my cm res_ref cr");
    $monitor ($ time ,,,, Ain ,,,, Bin ,,,, Ci ,,,, res_my ,,,,,, cm ,,,,,, res_ref ,,,, ,,, cr);
    # 400 $finish;
  end

```

```

end

initial
begin
Ain_r = 1;
# 50 Ain_r = 5;
# 50 Ain_r = 1;
# 50 Ain_r = 5;
# 50 Ain_r = 1;
# 50 Ain_r = 5;
# 50 Ain_r = 1;
# 50 Ain_r = 5;
end

initial
begin
Bin_r = 2;
# 100 Bin_r = 10;
# 100 Bin_r = 2;
# 100 Bin_r = 10;
end

initial
begin
Ci_r = 1'b0;
# 200 Ci_r = 1'b1;
end

assign Ain = Ain_r;
assign Bin = Bin_r;
assign Ci = Ci_r;

endmodule

```

Зверніть увагу на підключення модулів в даному файлі і використання системних функцій.

4. Скомпілюйте всі програми. Після успішної компіляції перейдіть в режим моделювання. В якості основного файлу для моделювання вкажіть тестовий файл.

Відкрийте графічне вікно і додайте до нього сигнали, що перевіряються. Запустіть проект на моделювання. Дайте відповідь НІ на запитання, чи хочете Ви закінчити роботу з симулятором. Перевірьте отримані результати:

3. Завдання до роботи

Спроектувати арифметико-логічний пристрій (вхідні дані - восьмирозрядні) з такими функціями:

$$\begin{aligned}
 &\text{not } (A + B), \\
 &\text{not } (A * B), \\
 &A + B + 1, \\
 &(A + \text{not}B) + 1 .
 \end{aligned}$$

Для перевірки роботи пристрою створити тестовий файл.